

KARNATAKA RESIDENTIAL EDUCATIONAL INSTITUTIONS SOCIETY

ICT Class 10 - Formative Assessment 3

Formative Assessment (FA-3) - Answer Key

Class: Class 10

Assessment Type: Formative Assessment (FA-3)

Total Marks: 30

Duration: 90 minutes

Topics Covered: Python Programming - Part 1

ICT Teaching Resources Portal

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: MCQ (5 x 1 = 5 marks)

- Answer:** a - import module
 - Explanation:** Python uses the import keyword to import modules. include, require, and using are not valid Python syntax.
- Answer:** b - <class 'list'>
 - Explanation:** [] creates an empty list in Python. The type() function returns <class 'list'>.
- Answer:** a - readline()
 - Explanation:** readline() reads one line from a file. There is no getLine(), fetchLine(), or line() method.
- Answer:** a - Opens file and closes it automatically after the block
 - Explanation:** The with statement ensures the file is properly closed after the block, even if an exception occurs.
- Answer:** b - ZeroDivisionError
 - Explanation:** Python raises ZeroDivisionError when attempting to divide by zero.

Section II: Short Answer (5 x 2 = 10 marks)

- Output and explanation of filter code**
 - Answer:** Output: [30, 40, 50]. The filter() function applies the lambda to each element. lambda x: x > 25 returns True for elements greater than 25. Only matching elements are included in the result list.
 - Marking:** 1 mark for correct output, 1 mark for explanation.
- Differentiate read(), readline(), readlines()**
 - Answer:** read() reads the entire file as a single string. readline() reads one line at a time. readlines() reads all lines and returns a list of strings. Use read() for small files, readline() for line-by-line processing, readlines() when you need a list.
 - Marking:** 0.5 marks for each method, 0.5 marks for use cases.
- Character frequency function**
 - Answer:**

```
def char_frequency(s):  
    freq = {}  
    for char in s:  
        freq[char] = freq.get(char, 0) + 1  
    return freq  
  
# Test  
print(char_frequency("hello")) # {'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

- **Marking:** 1 mark for function definition, 1 mark for correct logic.

4. Try-except for file handling

- **Answer:** try-except blocks handle runtime errors gracefully. Example:

```
try:
    f = open("data.txt", "r")
    content = f.read()
    f.close()
except FileNotFoundError:
    print("File not found!")
except PermissionError:
    print("No permission to read file!")
```

- **Marking:** 1 mark for purpose, 1 mark for example.

5. Syntax error vs Runtime error

- **Answer:** Syntax errors occur when code violates Python grammar (detected before execution). Example: `print("hello"` (missing parenthesis). Runtime errors occur during execution. Example: `10 / 0` causes `ZeroDivisionError`.
- **Marking:** 1 mark for each type with example.

Section III: Long Answer (5 x 1 = 5 marks)

1. Student Records Management Program

- **Answer:**

```
# Write student records
students = [
    {"name": "Rahul", "roll": 1, "marks": 85},
    {"name": "Priya", "roll": 2, "marks": 92},
    {"name": "Amit", "roll": 3, "marks": 78},
    {"name": "Sneha", "roll": 4, "marks": 88},
    {"name": "Vijay", "roll": 5, "marks": 71}
]

with open("students.txt", "w") as f:
    for s in students:
        f.write(f"{s['name']},{s['roll']},{s['marks']}\n")

# Read and display sorted by marks
with open("students.txt", "r") as f:
    lines = f.readlines()

data = []
for line in lines:
    name, roll, marks = line.strip().split(",")
    data.append({"name": name, "roll": int(roll), "marks": int(marks)})

data.sort(key=lambda x: x["marks"], reverse=True)

print("Students sorted by marks:")
for s in data:
    print(f"{s['name']} - Roll: {s['roll']} - Marks: {s['marks']}")

# Find highest marks
topper = max(data, key=lambda x: x["marks"])
print(f"\nTopper: {topper['name']} with {topper['marks']} marks")
```

```
# Count above 75%
count = sum(1 for s in data if s["marks"] > 75)
print(f"Students above 75%: {count}")
```

- **Marking:** 1 mark for file writing, 1 mark for reading, 1 mark for sorting, 1 mark for topper, 1 mark for counting.

Part B: Hands-on Practical Lab Task (5 marks)

File Statistics Program

```
import collections

def analyze_file(filename):
    try:
        with open(filename, 'r') as f:
            content = f.read()

        lines = content.split('\n')
        words = content.split()
        chars = len(content)
        unique_words = set(words)

        # Word frequency
        word_count = collections.Counter(words)
        top5 = word_count.most_common(5)

        # Write summary
        with open('summary.txt', 'w') as f:
            f.write(f"Total Lines: {len(lines)}\n")
            f.write(f"Total Words: {len(words)}\n")
            f.write(f"Total Characters: {chars}\n")
            f.write(f"Unique Words: {len(unique_words)}\n")
            f.write(f"Top 5 words: {top5}\n")

        print(f"Lines: {len(lines)}, Words: {len(words)}, Chars: {chars}")
        print(f"Top 5: {top5}")

    except FileNotFoundError:
        print("File not found!")
    except Exception as e:
        print(f"Error: {e}")

analyze_file("input.txt")
```

- **Marking:** 1 mark for file reading, 1 mark for counting, 1 mark for frequency analysis, 1 mark for file writing, 1 mark for exception handling.

Part C: Notebook Maintenance (5 marks)

1. Neatness and organisation of notebook (2 marks)
2. Completion of all assigned classwork and homework (2 marks)
3. Proper headings, dates, and indexing (1 mark)